ANÁLISIS DE COMPLEJIDAD
DONOVAN SALGUERO
JUAN DAVID MUÑOZ COLMENARES

Clasificación de fragmentos, comparación de crecimiento y reflexión sobre los resultados

1. Clasificación de los fragmentos

La notación Big O describe cómo cambia el trabajo de un algoritmo cuando aumenta el tamaño de los datos de entrada.

Fragmento	Operaciones	Big O	Justificación
Acceder a vector[5]	1	O(1)	Se consulta una posición concreta; el tamaño del arreglo no cambia el cos to.
Recorrer un arreglo	n	O(n)	Se revisa cada elemento una vez, por lo que el trabajo crece con n.
Dos ciclos anidados	n × n = n²	O(n²)	El ciclo interno realiza n vueltas por cada iteración del externo.
Dividir n entre 2 repetidamente	log■ n	O(log n)	En cada paso el problema se reduce aproximadamente a la mitad.
Ciclo n + reducción logarítmica	n × log■ n	O(n log n)	El ciclo principal se repite n veces y dentro se hacen pasos logarítmicos.

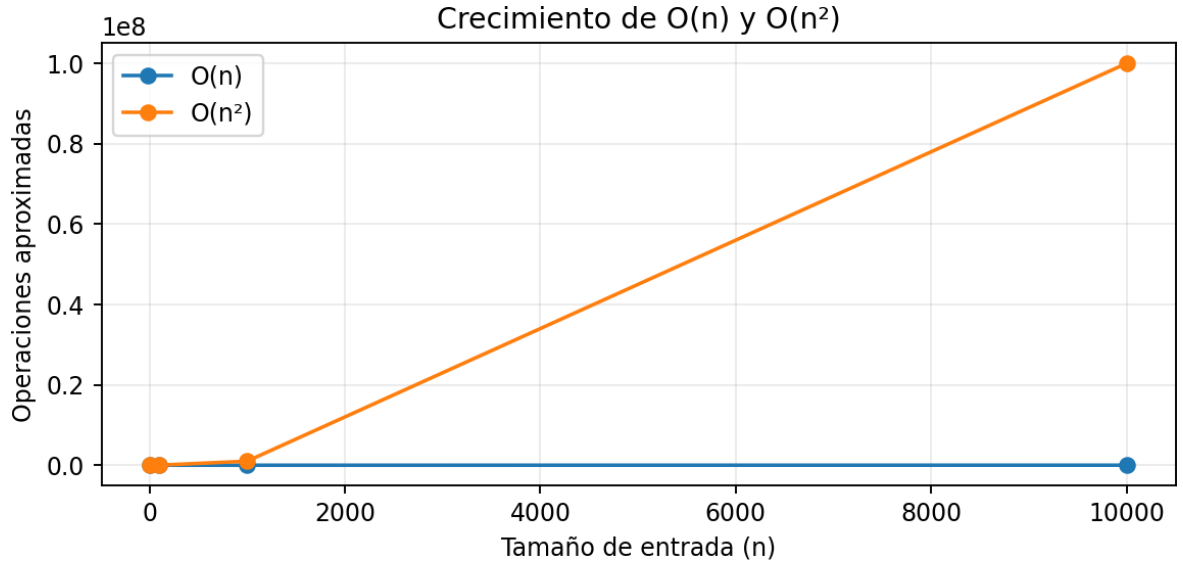
2. Comparación teórica: O(n) y O(n²)

La siguiente tabla muestra la cantidad aproximada de operaciones para varios tamaños de entrada.

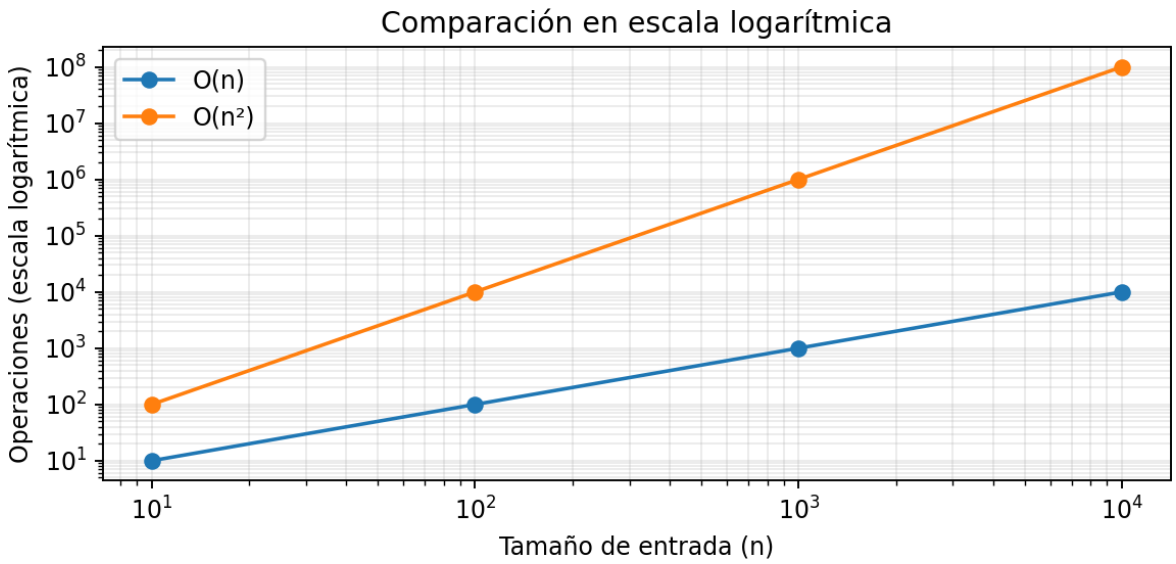
n	O(n)	O(n²)	Relación O(n²)/O(n)
10	10	100	10
100	100	10.000	100
1.000	1.000	1.000.000	1.000
10.000	10.000	100.000.000	10.000

Observaciones

- Al pasar de n = 10 a n = 100, O(n) aumenta 10 veces y O(n²) aumenta 100 veces.
- Con n = 10.000, el modelo lineal representa 10.000 operaciones, mientras que el cuadrático llega a 100.000.000.
- La relación entre ambos modelos es n, por lo que la diferencia se amplía cuando crece la entrada.
- En una escala lineal, O(n²) aumenta mucho más rápido; la escala logarítmica facilita comparar ambas tendencias.



3. Comparación gráfica



La escala logarítmica permite observar las dos tendencias en un mismo gráfico. No cambia los datos: únicamente presenta los valores usando una escala distinta en los ejes.

4. Preguntas de análisis

1. ¿Cuál es la diferencia entre tiempo de ejecución y complejidad Big O?

El tiempo de ejecución es la duración que se mide en un computador y puede variar por el procesador, la memoria y otros procesos. Big O representa cómo crece el trabajo al aumentar n, sin depender de una máquina específica.

2. ¿Por qué O(1) se considera eficiente?

Porque el trabajo permanece constante aunque aumente el tamaño de la entrada. Consultar una posición concreta puede tener un costo similar en un arreglo pequeño o en uno muy grande.

3. ¿Qué sucede con O(n) cuando se duplica la entrada?

En teoría, el trabajo se duplica aproximadamente. En el experimento, al pasar de 5.000 a 10.000 elementos, el tiempo cambió de 162.000 a 316.900 ns, casi el doble.

4. ¿Qué se espera de O(n²) al duplicar n?

Teóricamente, el trabajo se cuadruplica porque $(2n)^2 = 4n^2$. En las mediciones, el tiempo pasó de 6.618.100 a 14.656.200 ns, unas 2,2 veces. Las diferencias pueden relacionarse con las condiciones de ejecución.

5. ¿Por qué los ciclos anidados suelen ser O(n²)?

Por cada una de las n vueltas del ciclo externo, el ciclo interno puede realizar otras n iteraciones. El total es $n \times n = n^2$. Para n = 10.000, el ejemplo alcanza 100.000.000 de incrementos.

6. ¿Cómo se identifica un algoritmo O(log n)?

En cada paso, el tamaño del problema se reduce a la mitad o a una fracción constante. Un ejemplo es dividir n entre 2 repetidamente.

7. ¿Cuál algoritmo presentó el mayor crecimiento?

El algoritmo B, de complejidad O(n²), tuvo el comportamiento más costoso. En todos los tamaños medidos su tiempo fue mayor que el de O(n); con n = 10.000 tardó cerca de 46 veces más.

8. ¿Por qué los tiempos pueden cambiar entre ejecuciones?

Influyen el procesador, la memoria disponible, el sistema operativo, los procesos activos y las optimizaciones de Java (JIT). Por eso, los resultados reales no siempre siguen exactamente la proporción teórica.

9. ¿Qué complejidad elegiría para procesar diez millones de registros?

Preferiría O(n) o, dependiendo del problema, O(n log n), porque ambas opciones crecen de forma más manejable que O(n²) cuando hay muchos datos.

10. ¿Por qué se debe analizar la eficiencia antes de elegir un algoritmo?

Porque un programa puede funcionar bien con pocos datos y volverse demasiado lento cuando la información aumenta. La eficiencia afecta el tiempo de respuesta y los recursos que necesita la aplicación.

5. Conclusiones

La actividad permitió reconocer que un programa no solo debe entregar el resultado correcto: también es importante considerar el trabajo que necesita para conseguirlo. En las pruebas, con 10.000 elementos, O(n²) registró 14.656.200 ns y O(n) registró 316.900 ns; el algoritmo cuadrático tardó aproximadamente 46 veces más.

Los tiempos medidos tampoco siguieron siempre las proporciones teóricas. Por ejemplo, el resultado de O(n²) para 1.000 elementos fue mayor que el de 5.000. Esto puede explicarse por factores del computador y por las optimizaciones de la JVM. Big O no indica segundos exactos, sino la tendencia del crecimiento del trabajo.

En conclusión, antes de escoger un algoritmo conviene pensar en la cantidad de datos que tendrá que procesar. Una solución que funciona con pocas entradas puede dejar de ser práctica cuando el sistema crece. Analizar la complejidad ayuda a desarrollar aplicaciones más eficientes.